MIT 8103 Advanced Database Systems: Portfolio Summary

Name: Daniel Jacob | **Student ID:** 301917531 | **Case Study:** Hotel Reservation System
GitHub Repository: https://github.com/Charis0109/Daniel_Jacob_301917531

Case study

The case study for this portfolio was the design and development of a Hotel Reservation System database. The purpose of the system was to manage hotel guests, rooms and reservations while maintaining accurate booking records and preventing double-booking.

The database was designed around three main entities: `guest`, `room` and `reservation`. The system supports the recording of guest details, room information, reservation dates and booking statuses. It also considers important hotel operations such as booking confirmation, cancellation, check-in and check-out.

The project provided an opportunity to apply database concepts to a practical business problem. PostgreSQL 16 was used for the implementation, while GitHub and GitHub Codespaces supported project storage and development.

Portfolio 1: Database Design and Modelling

Portfolio 1 focused on database design and modelling for the Hotel Reservation System. I identified the main entities, their attributes, relationships and business rules. I also worked on the Entity Relationship Diagram (ERD) and normalisation to organise the data and reduce unnecessary duplication. The database contains three main tables: `guest`, `room` and `reservation`. Primary keys and foreign keys were used to identify records and maintain relationships. Other constraints, including `NOT NULL`, `UNIQUE` and `CHECK`, were applied to enforce data quality.

One important design decision was the use of a PostgreSQL exclusion constraint to prevent overlapping active reservations for the same room. This was important because it helps prevent two guests from booking the same room for overlapping dates. Cancelled reservations were allowed to overlap with active reservations because a cancelled booking should not block a valid booking.

I tested the database constraints using sample data and a constraint-testing script. All 16 test cases behaved as expected. Tests 1–15 rejected invalid entries, while test 16 accepted an overlapping reservation because the existing booking was cancelled. The recorded final test data contained 3 guests, 4 rooms and 6 reservations.

This portfolio helped me understand how database design decisions and constraints can reflect real business rules.

Portfolio 2: Query Processing and Optimisation

Portfolio 2 covered query processing and optimisation. This area of database management focuses on how database systems execute SQL queries and how queries can be improved to retrieve information more efficiently.

The portfolio folder is available in my GitHub repository. However, the exact SQL queries, execution plans and measured before-and-after timings are not included in the materials available for this summary.

The performance comparison requires the actual outputs from the original and optimised queries.

The main lesson from this portfolio was the importance of considering how a query is executed, rather than focusing only on whether it returns the correct result.

Portfolio 3: Transactions and Concurrency

Portfolio 3 focused on database transactions, particularly the use of `COMMIT` and `ROLLBACK` to control changes to database records. I carried out a transaction that changed reservation 1 from Pending to Confirmed. The update affected one record and was successfully committed. When I repeated the same conditional update, PostgreSQL returned `UPDATE 0` because the reservation was no longer Pending.

I also tested a second transaction using reservation 2. The transaction first changed the status from Confirmed to Pending. A subsequent attempt to assign an invalid status, `Invalid`, violated the reservation status check constraint. The transaction was rolled back. This exercise demonstrated that a transaction can contain multiple operations and that a failure can result in the earlier changes being undone when the transaction is rolled back. It helped me understand why transactions are important for maintaining database consistency.

Portfolio 4: NoSQL and Advanced Data Models

Portfolio 4 covered NoSQL databases and advanced data models. This topic extends database knowledge beyond the traditional relational model and introduces alternative ways of storing and managing data.

The broader learning objective was to understand that different applications have different data requirements and that database models should be selected based on the structure and use of the information being stored.

Portfolio 5: Distributed and Cloud Database Exercise

Portfolio 5 focused on distributed and cloud database concepts. These concepts consider how databases can be deployed across different environments and how availability, scalability, data consistency and reliability affect database architecture.

The portfolio folder is included in my repository. No specific deployment, configuration or performance result is claimed here because the practical evidence has not been provided for this summary. A distributed or cloud-based design should be evaluated against the requirements of the application. Any proposed architecture or test that was not implemented should be identified as a proposal rather than a completed practical exercise.

Evidence

The GitHub repository contains the portfolio folders and provides a record of the project work.

Portfolio 1: Database Design and Modelling

Portfolio 2: Query Processing and Optimisation

Portfolio 3: Transactions and Concurrency

Portfolio 4: NoSQL and Advanced Data Models

Portfolio 5: Distributed and Cloud Database Exercise

The Portfolio 3 screenshots include `Daniel_Jacob_301917531_P3_commit.png` and `Daniel_Jacob_301917531_P3_rollback.png`. Portfolio 1 evidence includes the design, ERD, normalisation, schema, sample data and constraint-testing materials.

Individual reflection

One of the most important things I learnt during this course is that I can learn anything I put my mind to. I initially found using GitHub challenging because it was my first time working with it. I had to understand how repositories, files and commits work. Rather than giving up when I encountered difficulties, I learnt from fellow students, observed how they approached their work and applied what I learnt to my own project.

Another important learning experience was using PostgreSQL constraints to enforce business rules. In particular, implementing an exclusion constraint to prevent overlapping active room reservations helped me understand how database features can solve practical business problems. Overall, the course has improved my confidence in learning new technical concepts, working with database tools and documenting my practical work. It has also taught me the importance of patience, consistency and asking questions when I encounter something I do not understand.

References

[1] PostgreSQL Global Development Group. PostgreSQL 16: Constraints. https://www.postgresql.org/docs/16/ddl-constraints.html
[2] PostgreSQL Global Development Group. PostgreSQL 16: Using EXPLAIN. https://www.postgresql.org/docs/16/using-explain.html
[3] PostgreSQL Global Development Group. PostgreSQL 16: Transaction Isolation. https://www.postgresql.org/docs/16/transaction-iso.html
[4] MongoDB. Data Modeling in MongoDB. https://www.mongodb.com/docs/manual/data-modeling/ Accessed 28 October 2026.

AI-use declaration

I used ChatGPT to support my learning during the portfolio exercises. I used it to understand SQL concepts, explain database operations and troubleshoot errors encountered while working on the exercises. I reviewed the explanations and applied the guidance while carrying out the practical SQL tasks using PostgreSQL.

ChatGPT was used as a learning and troubleshooting aid. I remained responsible for carrying out the practical work and checking the results.